
文件编号：JWMATH—YF—202509

时间：2025 年 09 月 08 日

发送：泾渭数联研发中心

抄送：泾渭数联

来源：泾渭数联总经办

主题：关于颁布《西安泾渭数联信息技术有限公司研发人员（软件开发规范）》的通知

经公司研究决定，颁布《西安泾渭数联信息技术有限公司研发人员（软件开发规范）》（以下简称《规范》）请各位员工自习学习并遵照执行，本规范适用于研发人员开发工作考核。

本《规范》自发布之日起执行，原有代码开发规范文件终止执行，凡与本《规范》冲突的条款，以本规范为准。

本《规范》适用对象为西安泾渭数联信息技术有限公司研发中心全体员工。

本《规范》解释权归西安泾渭数联信息技术有限公司总经办所有。附件：《西安泾渭数联信息技术有限公司研发人员（软件开发规范）》

西安泾渭数联信息技术有限公司

西安泾渭数联信息技术有限公司
软件开发规范

文件状态：	文件标识：	
☐ 草稿	当前版本：	V1.0
☒ 正式发布	作 者：	西安泾渭数联信息技术有限公司
☐ 正在修改	完成日期：	2025-09-08

目 录

一、总则 3

二、通用规范 3

三、语言专项规范 4

 1. Java 规范 4

 2. HTML 规范 6

 3. JavaScript 规范 7

 4. Python 规范 8

 5. SQL 规范 10

四、测试规范 18

五、文档规范 18

一、总则

1. **目的：**规范开发流程，提高代码质量，促进团队协作，降低维护成本
2. **适用范围：**公司所有研发人员
3. **基本原则：**清晰可读、简洁高效、安全可靠

二、通用规范

1. 命名规范

- 禁止使用拼音、拼音首字母混合命名
- 禁止使用无意义命名（如 a、temp、test）
- 命名需准确反映功能或含义，长度适中（不超过四个单词）
- 包类命名：每一个包的名称总是小写，如：com.jwmath.fundmodel
- 前端文件命名(html、js、css、vue 等)：全小写，**单词间用 ‘-’ 隔开**，fund-model.js
- 前端样式选择器命名，除 id 用小驼峰命名外，其他用全小写，单词间用 ‘-’ 隔开。
如 `<input id="userId" />` `<input class="user-id" />`

2. 代码风格

- 缩进统一（Java/JS 使用 4 空格，Python 使用 4 空格，HTML 使用 2 空格）
- 一行只写一条语句，代码长度不超过 120 字符
- 合理使用空行分隔逻辑块，提高可读性

3. 注释要求

- 类、方法必须有注释，说明功能、参数、返回值、创建人
- 修改别人代码，注明：修改人、修改描述
- 复杂逻辑需添加行注释
- 注释需与代码同步更新，避免误导

4. 代码长度

- java、python、JavaScript 单方法长度**不可超过 100 行**，包含空行，不含注释
- Java、python 单类长度**不可超过 1000 行**

5. 版本控制

- 提交信息格式：[类型] 描述（如：[修复] 解决登录超时问题）
- **功能开发完成后及时提交，单次提交代码量不宜过大**

- 保护敏感信息，禁止提交密码、密钥等

6. 测试

- 这里的测试针对开发人员，包含代码级别的单元测试和集成测试
- 开发人员完成功能点开发完成后，需写单元测试，覆盖率不低于 70%，配合 jcoco、coverage 等工具
- 可利用 AI 工具生成测试代码，但必须测试通过

7. 项目管理

- 项目内部必须保证 jdk、python 虚拟环境、数据库、中间件等版本完全一致
- 开发人员不可私自引入三方库、中间件，必要时需向项目负责人沟通、确认
- 开发人员每天下班前提交可执行代码，如无法提交，说明原因
- 一个模块完整开发、验证完成后，模块开发人员须补充完整的文档（设计、数据库设计、操作手册）

三、语言专项规范

1. Java 规范

1.1 命名规范

- 类名：使用 UpperCamelCase，如 `UserController`、`OrderServiceImpl`
- 方法名：使用 lowerCamelCase，动词开头，如 `createUser()`、`calculateTotalPrice()`
- 变量名：使用 lowerCamelCase，如 `userName`、`orderList`
- 常量名：使用 UPPER_SNAKE_CASE，如 `MAX_RETRY_COUNT = 3`、`DEFAULT_TIMEOUT = 5000`
- 包名：全小写，使用公司域名反转 + 模块名，如 `com.jwmath.user.service`
- 接口名：使用 UpperCamelCase，前缀 `I`（如 `IUserService`）团队需统一
- 枚举类：类名使用 UpperCamelCase，枚举值使用 UPPER_SNAKE_CASE，如 `enum OrderStatus { CREATED, PAID, SHIPPED }`

1.2 代码结构

- 类成员顺序：静态变量 → 成员变量 → 构造方法 → 静态方法 → 成员方法
- 方法参数：不超过 5 个，超过时使用对象封装
- 方法长度：单个方法代码不超过 100 行，复杂逻辑需拆分
- 类长度：单个类代码不超过 1000 行，过大时按职责拆分

1.3 语法规范

- 异常处理：
 - 必须捕获具体异常，禁止 `catch (Exception e)`
 - 异常信息需包含具体上下文，如 `throw new IllegalArgumentException("用户 ID 不能为空：" + userId)`
 - 禁止空 `catch` 块，至少记录日志
- 集合处理：
 - 初始化指定容量： `List<User> users = new ArrayList<>(10)`
 - 遍历使用增强 `for` 循环： `for (User user : users) {}`
 - 集合判空使用 `CollectionUtils.isEmpty(collection)`
- 注释规范：

```
/**
 * 用户服务实现类，处理用户 CRUD 操作
 * @author 张三
 * @date 2025-08-24
 */
```

```
/**
 * 根据 ID 查询用户
 * @param userId 用户 ID
 * @return 用户信息，不存在返回 null
 * @throws IllegalArgumentException 当 userId 为 null 时抛出
 */
```

- 类注释：包含功能描述、作者、创建日期
- 方法注释：包含功能、参数、返回值、异常

1.4 性能与安全

- 字符串拼接：循环中使用 `StringBuilder`，避免 `+` 运算符
- 资源释放：IO 流、数据库连接等必须在 `try-with-resources` 中使用

- 线程安全：共享变量需加锁或使用线程安全容器
- 避免魔法值：使用常量或枚举替代硬编码数值
- 安全校验：所有外部输入必须进行校验，防止 XSS、SQL 注入

1.5 框架规范

- Spring 框架：
 - 服务层命名后缀为 `Service`，实现类后缀为 `ServiceImpl`
 - 控制器命名后缀为 `Controller`
 - 事务注解 `@Transactional` 加在业务层，指定 `rollbackFor`

2. HTML 规范

2.1 结构规范

- DOCTYPE 声明：必须使用 `<!DOCTYPE html>`
- 语义化标签：
 - 使用 `<header>`、`<nav>`、`<main>`、`<section>`、`<footer>` 等替代无意义的 `<div>`
 - 使用 `<button>` 而非样式化的 `<a>` 或 `<div>` 处理点击事件
- 嵌套规则：
 - 块级元素可包含行内元素，反之不允许
 - `<p>` 标签内不能包含块级元素
 - 列表结构需完整：`<ul>`/`<ol>` 包含 `<li>`，`<dl>` 包含 `<dt>` 和 `<dd>`

2.2 属性规范

- 标签和属性名：全部小写
- 属性值：统一使用双引号包裹
- 必选属性：
 - `<img>` 必须包含 `alt` 属性
 - `<a>` 需指定 `href`，空链接使用 `href="javascript:void(0)"`
 - 表单元素需有 `name` 或 `id`，并与 `<label>` 关联

2.3 样式与脚本

- 样式：优先使用外部 CSS，避免行内样式

- 脚本: `<script>`放在`<body>`底部, 或使用 `defer`/`async` 属性
- 注释: 复杂结构需添加注释, 格式 `<!-- 导航菜单开始 -->`

3. JavaScript 规范

3.1 变量与类型

- 声明:
 - 优先使用 `const`, 变量需修改时使用 `let`, 禁止使用 `var`
 - 一行只声明一个变量: `const name = 'Tom'; let age = 20;`
- 类型:
 - 字符串使用单引号: `const message = 'hello'`
 - 对象初始化: `const user = { name: 'Tom', age: 20 }` (尾部无逗号)
 - 数组初始化: `const fruits = ['apple', 'banana']`

3.2 函数与箭头函数

- 函数声明: 优先使用箭头函数处理回调

```
// 推荐
const sum = (a, b) => a + b;
// 复杂函数
const calculate = (x, y) => {
  const result = x * y;
  return result;
};
```

- 参数:
 - 传递多个参数时使用对象解构: `const func = ({ name, age }) => {}`
 - 设置默认值: `const greet = (name = 'Guest') => {}`

3.3 异步处理

- 优先使用 `async/await`, 避免回调地狱


```
// 推荐

const fetchData = async () => {
  try {
    const response = await fetch(url);
    const data = await response.json();
    return data;
  } catch (error) {
    console.error('请求失败:', error);
  }
};
```

3.4 代码风格

- 分号：语句结束必须加分号
- 比较：使用 `===` 和 `!==`，避免 `==` 和 `!=`
- 括号：条件语句必须带大括号，即使单行

```
// 推荐

if (isValid) {
  doSomething();
}

// 禁止

if (isValid) doSomething();
```

4. Python 规范

4.1 命名规范

- 函数名、变量名：使用 snake_case，如 `user_name`、`calculate_price()`
- 类名：使用 UpperCamelCase，如 `UserManager`
- 常量：使用 UPPER_SNAKE_CASE，如 `MAX_RETRY = 3`
- 模块名：全小写，简短，如 `user_service.py`

4.2 代码格式

- 缩进：严格使用 4 个空格
- 空行：
 - 类定义后空两行
 - 函数定义后空一行
 - 逻辑块之间空一行
- 行长度：不超过 120 个字符，过长需换行

```
# 推荐  
  
result = long_function_name(var_one, var_two,  
                             var_three, var_four)
```

4.3 导入规范

- 导入顺序：标准库 → 第三方库 → 本地应用 / 库
- 禁止使用 `from module import *`
- 分组导入：

```
# 标准库  
import os  
import sys  
  
# 第三方库  
import requests  
from flask import Flask  
  
# 本地库  
from .models import User
```

4.4 注释规范

- 文档字符串：使用三双引号，包含功能、参数、返回值，**注意与 java 的区别。**

```
def calculate_area(radius):  
    """计算圆的面积  
  
    Args:  
        radius (float): 圆的半径  
  
    Returns:  
        float: 圆的面积  
  
    Raises:  
        ValueError: 当半径为负数时抛出  
    """
```

- 行内注释：与代码间隔两个空格

```
total = price * quantity # 计算总价
```

4.5 语法规范

- 条件判断：避免单行 if 语句
- 列表推导：简单逻辑可用，复杂逻辑应拆分
- 异常处理：明确捕获特定异常，不使用 `except Exception`
- 类型提示：关键函数添加类型提示

```
def add(a: int, b: int) -> int:  
    return a + b
```

5. SQL 规范

说明：sql 语句可读性比较差，特别是当 sql 行数超过二三十行的时候，调试也比较 java、python 等语言困难，sql 规范之前，数据库设计文档就特别必要，特别是对新人。下列规范建立在对数据库设计了解的情况下。

5.1 命名规范

- **表名**: 使用 snake_case, 以复数形式表示数据集合, 如 `users`、`order_items`、`product_categories`, 禁止使用缩写 (如 `ord` 需改为 `orders`)
- **列名**: 使用 snake_case, 明确字段含义, 如 `user_name`、`order_amount`、`created_at`, 避免使用 SQL 关键字 (如 `order` 需改为 `order_no`、`desc` 需改为 `description`)
- **主键**: 统一命名为 `id` (单表主键), 联合主键命名为 `{关联表名}_id`, 如 `user_id`、`role_id` (多对多关联表)
- **外键**: 格式为 `{关联表名}_id`, 如 `user_id` (关联 `users` 表)、`parent_id` (自关联表, 如分类表)
- **索引**: 前缀 `idx_` + 表名 + 列名 (联合索引按字段优先级排序), 如 `idx_users_email` (单字段索引)、`idx_orders_user_id_created_at` (联合索引)
- **视图**: 前缀 `v_` + 功能描述, 如 `v_active_users` (活跃用户视图)、`v_order_statistics` (订单统计视图)
- **存储过程**: 前缀 `sp_` + 动作 + 功能, 如 `sp_update_user_status` (更新用户状态)、`sp_generate_order_no` (生成订单号)
- **函数**: 前缀 `fn_` + 功能, 如 `fn_calculate_tax` (计算税费)、`fn_format_date` (格式化日期)

5.2 格式规范

- **关键字**: 全部大写, 如 `SELECT`、`FROM`、`WHERE`、`JOIN`、`GROUP BY`, 提升可读性
- **缩进与换行**: 子句 (`SELECT`/`FROM`/`WHERE` 等) 单独换行, 子句内字段 / 条件缩进 4 个空格

-- 推荐

```
SELECT
    user_id,
    user_name,
    email,
    created_at
FROM
    users
```

```
WHERE

    status = 1  -- 1 表示正常状态

    AND created_at BETWEEN '2025-01-01' AND '2025-08-24'

ORDER BY

    created_at DESC

LIMIT 10;
```

- **逗号位置：**字段后加逗号，放在行尾（而非行首），便于新增 / 删除字段时减少代码修改
- **注释格式：**
 - 单行注释：使用 `-- 注释内容`，用于解释单行逻辑或字段含义
 - 块注释：使用 `/* 多行注释内容 */`，用于解释复杂 SQL 脚本（如存储过程、批量更新逻辑）
 - 复杂查询必须添加注释，说明业务场景（如 `-- 统计近 30 天各部门新增用户数`）

5.3 表设计规范

- **必含字段：**每张业务表必须包含 4 个基础字段，确保数据可追溯、可管理

```
CREATE TABLE users (

    id BIGINT PRIMARY KEY AUTO_INCREMENT COMMENT '用户主键 ID，自增',

    create_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间，
    自动填充当前时间',

    update_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP COMMENT '更新时间，修改时自动更新',

    is_delete TINYINT NOT NULL DEFAULT 0 COMMENT '逻辑删除标识：0-未删除，1-已删
    除'

    -- 其他业务字段...

) COMMENT = '用户基础信息表';
```

- **字段类型选择：**
 - 整数类型：按数据范围选择最小合适类型（`tinyint`：1 字节，`smallint`：2 字节，`int`：4 字节，`bigint`：8 字节），如 `status` 用 `tinyint`（1-5 个状态值）、`user_id` 用 `bigint`（支持海量用户）

- 字符串类型：明确长度，`varchar` 用于变长字符串（如 `user_name` 用 `varchar(50)`），`char` 用于定长字符串（如 `phone` 用 `char(11)`，手机号固定 11 位），禁止滥用 `varchar(255)`
- 时间类型：使用 `timestamp`（自动适配时区），如 `created_at`/`updated_at`，禁止使用 `datetime`（需手动维护时区）
- 约束设置：
 - 非必要字段不允许为 `NULL`，`NULL` 字段需设置默认值（如 `status` 默认 1、`remark` 默认空字符串''）
 - 唯一约束：重复数据需加 `UNIQUE`，如 `email`、`phone`（避免重复注册）
 - 外键约束：关联表需加外键（可选，若用 ORM 框架可在应用层维护），如 `orders` 表的 `user_id` 关联 `users` 表的 `id`

5.4 查询规范

- 禁止 `SELECT *`：明确指定所需字段，减少数据传输量，避免字段变更导致的异常

-- 推荐

```
SELECT id, user_name, email FROM users WHERE is_deleted = 0;
```

-- 不推荐

```
SELECT * FROM users WHERE is_deleted = 0;
```

- **WHERE 条件优化：**
 - 优先使用索引字段过滤（如 `id`、`user_id`、`created_at`），减少返回数据量
 - 避免在索引字段上使用函数，防止索引失效（如 `DATE(created_at) = '2025-08-24'` 需改为 `created_at BETWEEN '2025-08-24 00:00:00' AND '2025-08-24 23:59:59'`）
 - 等值条件（`=`）放在前面，范围条件（`BETWEEN`/`IN`）放在后面，如 `WHERE user_id = 123 AND created_at >= '2025-01-01'`
- **JOIN 操作规范：**

-- 推荐

```
SELECT
```

```
    u.user_name,
```

```
o.order_no,
o.order_amount
FROM
  users u -- 小表作为驱动表
INNER JOIN
  orders o ON u.id = o.user_id -- o.user_id 为索引字段
WHERE
  u.is_deleted = 0
  AND o.is_deleted = 0;
```

- 明确指定 JOIN 类型（INNER JOIN/LEFT JOIN/RIGHT JOIN），禁止省略（默认 INNER JOIN 易混淆）
 - 小表驱动大表：将数据量小的表作为驱动表（如用 users 表关联 orders 表），减少循环次数
 - JOIN 条件必须包含索引字段，如 u.id = o.user_id（u.id 为主键索引，o.user_id 为外键索引）
- 排序与分页：

```
-- 优化前（偏移量 10000，需扫描 10100 行）
SELECT * FROM orders WHERE is_deleted = 0 LIMIT 10000, 10;
-- 优化后（仅扫描 10 行，需 id 连续）
SELECT * FROM orders WHERE id > 10000 AND is_deleted = 0 LIMIT 10;
```

- 大表排序（数据量 > 10 万）需添加排序字段索引，避免 Using filesort（如 ORDER BY created_at DESC 需加 idx_users_created_at 索引）
 - 分页优化：大偏移量（OFFSET > 1000）避免使用 LIMIT offset, size，改用 ID 过滤（减少全表扫描）
- 子查询与 JOIN 转换：复杂子查询（如多层嵌套、关联多表）改为 JOIN 操作，提升性能

```
-- 子查询（不推荐，多层嵌套难维护）
SELECT user_name FROM users WHERE id IN (SELECT user_id FROM orders WHERE
```

```
order_amount > 1000);  
-- JOIN 改写（推荐，易维护且性能更优）  
SELECT DISTINCT u.user_name  
FROM users u  
INNER JOIN orders o ON u.id = o.user_id  
WHERE o.order_amount > 1000 AND u.is_deleted = 0;
```

5.5 写入与更新规范

- INSERT 操作:

```
-- 推荐（批量插入 3 条用户数据）  
INSERT INTO users (user_name, email, phone)  
VALUES  
    ('张三', 'zhangsan@test.com', '13800138000'),  
    ('李四', 'lisi@test.com', '13900139000'),  
    ('王五', 'wangwu@test.com', '13700137000');  
-- 不推荐（单条插入，执行效率低）  
INSERT INTO users (user_name, email, phone) VALUES ('张三', 'zhangsan@test.com',  
'13800138000');
```

- 明确指定字段名，不依赖表结构顺序（防止字段新增 / 删除导致数据错位）
- 批量插入使用 **VALUES** 多组值，减少 SQL 执行次数（单次批量不超过 1000 条）

- UPDATE 操作:

```
-- 推荐（指定 WHERE 条件，仅更新单条数据）  
UPDATE users  
SET user_name = '张三三', updated_at = NOW()  
WHERE id = 123 AND is_deleted = 0;  
-- 危险（无 WHERE 条件，全表更新）  
UPDATE users SET user_name = '张三三';
```

- 必须包含 **WHERE** 条件，禁止全表更新（若需全表更新，需加业务负责人审批）
- 避免更新主键字段，禁止更新 **created_at**（创建时间）字段

- 删除操作:

```
-- 推荐（逻辑删除）
UPDATE users SET is_deleted = 1, updated_at = NOW() WHERE id = 123;
-- 谨慎使用（物理删除，数据不可恢复）
DELETE FROM users WHERE id = 123;
```

- 优先使用逻辑删除（更新 `is_deleted` 字段），保留数据追溯能力；物理删除（`DELETE`）需经审批（如日志表清理）

- 事务控制:

```
BEGIN; -- 开启事务
-- 1. 插入订单数据
INSERT INTO orders (order_no, user_id, order_amount) VALUES ('ORD20250824001',
123, 999.00);
-- 2. 扣减商品库存（需判断库存是否充足）
UPDATE products SET stock = stock - 1 WHERE id = 456 AND stock >= 1;
-- 3. 检查库存扣减结果，失败则回滚
IF ROW_COUNT() = 0 THEN
    ROLLBACK; -- 回滚事务
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '商品库存不足';
END IF;
COMMIT; -- 提交事务
```

- 多表写入 / 更新（如“创建订单 + 扣减库存”）必须使用事务，保证原子性（要么全成功，要么全失败）

5.6 安全规范

- 防止 SQL 注入:

```
// Java (PreparedStatement)
String sql = "SELECT * FROM users WHERE user_name = ? AND is_deleted = 0";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, userName); // 参数化传值，避免注入
```

```
# Python (MySQLdb)

cursor.execute(

    "SELECT * FROM users WHERE user_name = %s AND is_deleted = 0",

    (userName,) # 元组传参，禁止字符串拼接

)
```

- 必须使用参数化查询，禁止字符串拼接 SQL（各语言示例如下）
- 敏感数据保护：
 - 密码存储：使用 BCrypt、SHA256 等算法加密，禁止明文存储，如 `INSERT INTO users (user_name, password) VALUES ('张三', BCrypt_HASH('123456'))`
 - 敏感字段查询：禁止在日志中输出手机号、邮箱等敏感数据，如 `SELECT user_id, user_name FROM users`（不查 `phone/email`）
- 权限控制：
 - 应用数据库账号按“最小权限原则”分配：查询账号仅授予 `SELECT` 权限，读写账号授予 `SELECT/INSERT/UPDATE` 权限，禁止授予 `DROP/ALTER/TRUNCATE` 等高危权限
 - 禁止使用 `root` 账号直接连接应用

5.7 性能优化规范

- 索引优化：
 - 高频查询字段（`WHERE/JOIN/ORDER BY`）添加索引，如 `users` 表的 `email`（登录查询）、`orders` 表的 `user_id`（用户订单查询）
 - 避免过度索引：单表索引不超过 5 个（索引会降低写入性能），删除无用索引（如长期未使用的索引）
 - 字符串索引：长字符串（如 `remark`）可指定前缀索引，如 `idx_users_remark(50)`（仅索引前 50 个字符）
- SQL 执行优化：
 - 避免 `DISTINCT`（可通过 `GROUP BY` 替代，或优化 `JOIN` 逻辑减少重复）
 - 禁止在 `WHERE` 子句中使用 `NOT IN`（改用 `LEFT JOIN + IS NULL`），如：

```
-- 不推荐（NOT IN 可能导致全表扫描）

SELECT * FROM users WHERE id NOT IN (SELECT user_id FROM orders);
```

```
-- 推荐（LEFT JOIN + IS NULL）  
  
SELECT u.*  
  
FROM users u  
  
LEFT JOIN orders o ON u.id = o.user_id  
  
WHERE o.user_id IS NULL AND u.is_deleted = 0;
```

- 大表优化：
 - 数据量超过 1000 万的表需分区（按时间 / 范围），如 `orders` 表按 `created_at` 分区（每月一个分区）
 - 历史数据归档：将超过 1 年的冷数据（如日志、历史订单）归档到归档表，减少主表数据量

四、测试规范

1. 核心功能需编写单元测试，覆盖率不低于 70%
2. 新增功能需进行回归测试
3. 测试通过后才能部署到测试环境

五、文档规范

1. 交付产品必须包含《用户操作文档》、《设计文档》、《数据库设计文档》、《部署文档》
2. 接口设计可利用相关工具，组内可阅读即可，交付时可导出成可读的文档。每一个接口文档需包含参数说明、返回值、错误码
3. 特别说明：需求文档由产品经理输出，研发针对产品经理的输出作为研发的输入。另外，需求文档一旦输出，在本迭代不可变更。没有提供需求文档的，研发人员依然要按照以上交付要求提供研发阶段相关文档说明。